Operation Sentinel – Architecture, Threat Model, Protocol and Components

FSCT 8561 Midterm, Parts A and B

Part A – Architecture Design

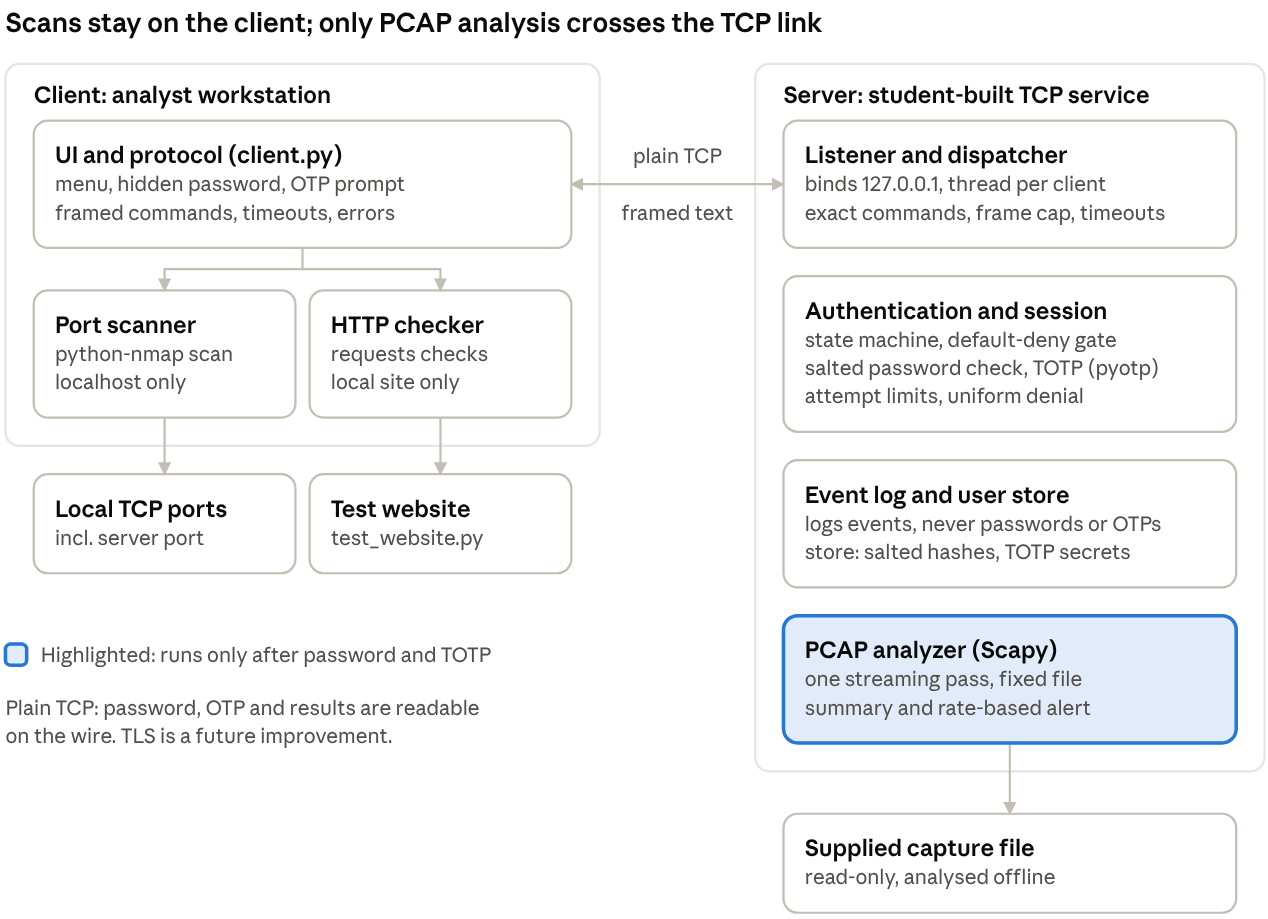
A.1 Overview

Operation Sentinel is built from the seven lab scripts, merged into **three client modules and four server modules** so that fewer pieces have to be written, run, and defended. An **analyst client** (`client.py`) talks to a **student-built TCP server** (`server.py`) over one persistent TCP connection on `127.0.0.1`. The port scan and HTTP check run **locally on the client**. The PCAP analysis runs **on the server** and is reachable only after password and TOTP authentication. All targets are limited to localhost or the instructor-provided test environment.

Three merges simplify the lab code:

- `auth_client.py` becomes one client module: the user interface and the message protocol live together in `client.py`.
- `auth_server.py` is split by job, not by lab: connection handling and command routing form the listener and dispatcher, and password checks, TOTP, and the connection state variables form one authentication and session module.
- `traffic_sniffer.py` and `anomaly_detector.py` become one PCAP analyzer, and `http_scanner.py` and `vulnerability_detector.py` become one HTTP checker.

Figure 1 – System architecture



architecture · 3 client modules, TCP link, 4 server modules

The client runs the port scan and HTTP check locally against localhost targets. The server accepts one framed TCP connection per client, enforces the session state machine, and runs the Scapy analysis only after password and TOTP succeed.

A.2 Component responsibilities

Component	Runs on	Built from (lab file)	Responsibility	Key security controls
UI and protocol (`client.py`)	Client	`auth_client.py`	Login prompt, OTP entry, menu, and results display. Frames and sends the commands `AUTH`, `OTP`, `PCAP`, `MSG`, `EXIT`; reads and checks each response; starts the local scan and HTTP check	Password read with `getpass` and never printed or stored; 4-byte length prefix and maximum frame size instead of the lab's bare `recv(1024)`; socket timeouts; input checked before sending; clean handling of server disconnect

Component	Runs on	Built from (lab file)	Responsibility	Key security controls
Port scanner (`scanner.py`)	Client	`namp_scanner.py`	Runs `python-nmap` (`-sT -sV`) on an authorized range that includes the server port; reports port, state, service, product, version	Target limited to `127.0.0.1` (the lab accepted any typed host); ports checked as 1–65535 with start ≤ end; 1000-port cap kept; fixed nmap arguments; scanner errors caught
HTTP checker (`http_checker.py`)	Client	`http_scanner.py` + `vulnerability_detector.py`	One pass against the local test website: observes each page (status, length, content type, `Server` header), checks the response headers `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, and `Referrer-Policy`, the cookie flags, the `Server` banner, and the redirect target; saves results to CSV	Fixed local base URL (`127.0.0.1:8765`); 5-second timeout; read-only `GET` requests plus one unsupported-method `POST` to check the error reply; redirects checked, never followed off-host; no exploit payloads; duplicate alerts suppressed; every finding worded as "manual review required"; connection failures recorded, not fatal
Listener and dispatcher (`server.py`)	Server	`auth_server.py` (socket setup and main loop)	Binds to localhost, accepts clients in a loop (one thread each), reads frames, parses the command, routes it to the right handler, sends the response, closes on `EXIT`	Bind to `127.0.0.1`; loop and threads replace the lab's single `accept()` so one probe cannot use up the server; exact command match instead of `startswith`; frame cap and idle timeout; field-by-field validation; per-connection error isolation
Authentication and session (`auth_session.py`)	Server	`auth_server.py` (`hash_password`, `verify_password`, `verify_otp`, state variables)	Keeps per-connection state (username, password verified, authenticated); checks the password, then the TOTP code with `pyotp`; tells the dispatcher whether a command is allowed	Salted PBKDF2-HMAC-SHA256 from `hashlib` replaces the lab's unsalted SHA-256; `hmac.compare_digest`; one `ACCESS_DENIED` for unknown user and wrong password; OTP must be six digits; attempt limit and lockout; OTP never printed (the lab printed it); default-deny for `PCAP` and `MSG`

Component	Runs on	Built from (lab file)	Responsibility	Key security controls
PCAP analyzer (`pcap_analyzer.py`)	Server	`traffic_sniffer.py` + `anomaly_detector.py`	One streaming pass with Scapy `PcapReader`: counts IPv4 TCP and UDP packets, lists the first 20 TCP/UDP packets, and flags any source IP sending more than 20 packets within a 5-second sliding window (one alert per IP); returns the summary to the client	File path fixed on the server, never taken from the client; one `PcapReader` pass (the lab sniffer opened the file twice, with `rdpcap` and `PcapReader`); only counts, addresses, ports, and packet evidence returned, no payload bytes; missing or corrupt file handled
Event log and user store (`security_log.py`, `users.json`)	Server	New (not in the labs)	Appends security events; holds username, salt, password hash, and TOTP secret	Logs time, address, username, event, outcome only; never passwords, OTPs, or secrets; replaces the lab's hard-coded `alice` password in source; restricted file permissions

A.3 Information flows and trust boundaries

Messages keep the lab's pipe-delimited text format (`COMMAND|field|field`). Each message is sent inside a length-prefixed frame so the receiver always knows where it ends.

Flow	From → To	Content	Notes
F1	Client → Server	TCP connect	Server creates a session in `CONNECTED` state
F2	Client → Server → Client	`AUTH|username|password`, reply `OTP_REQUIRED` or `ACCESS_DENIED`	Password is split with a field limit so it may contain `|`. Success moves the session to `PASSWORD_VERIFIED`. The password crosses the channel unencrypted
F3	Client → Server → Client	`OTP|123456`, reply `ACCESS_GRANTED` or `ACCESS_DENIED`	Success moves the session to `AUTHENTICATED`

Flow	From → To	Content	Notes
F4	Client → Server → Client	`PCAP`, reply `OK|<summary and alerts>`	Allowed only in `AUTHENTICATED`; the server runs the Scapy analysis and sends back text results
F5	Client → Server → Client	`MSG|text`, reply `OK|...`	Kept from the lab as a second protected command, used to demonstrate rejection of unauthorized requests
F6	Client → localhost TCP ports	Connect and service probes from `python-nmap`	Local to the client; the range includes the server's listening port, so the server also receives these probes
F7	Client → test website	HTTP requests from `requests`	Local to the client; results go to the console and a CSV file; the server is not involved
F8	Client → Server	`EXIT|` or disconnect	Session destroyed; event logged

How the parts connect. The only network link between two of your own programs is the client-to-server TCP connection (F1–F5 and F8). The port scanner and HTTP checker are not services and are not called with requests. They are Python modules imported by `client.py`, and each menu choice is an ordinary function call inside the client process, for example `run_port_scan(start, end)` and `run_http_checks()`. Each module then talks to its own target:

- **Menu: scan ports.** `client.py` calls the scanner module, which starts nmap. Nmap opens TCP connections to the localhost ports, including the server's port 5050.
- **Menu: HTTP check.** `client.py` calls the HTTP checker module, which uses `requests` to send HTTP `GET` requests to `127.0.0.1:8765` (`test_website.py`). This is the only place an HTTP request is made.
- **Menu: PCAP analysis.** `client.py` sends the `PCAP` command over the TCP protocol. The server's dispatcher checks the session state and only then calls the PCAP analyzer module, a function inside the server process. The client never reaches the analyzer directly.

Trust boundaries. (TB1) Client ↔ server: the server treats every byte from the client as untrusted input. (TB2) Client ↔ scan and HTTP targets: responses from targets are untrusted. (TB3) Server ↔ filesystem: the capture file, user store, and log are sensitive local assets.

Limitation to state explicitly. The channel is plain TCP. TOTP protects against reuse of a stolen password but gives no confidentiality or integrity. A passive observer can read the password, the OTP, and the PCAP results, so no confidentiality claim is made. TLS is listed as a future improvement.

A.4 Interaction model and rationale: stateful per connection

The authentication and session module holds one small state object per TCP connection. It turns the lab's loose variables (`username`, `password_verified`, `authenticated`, `connected`) into a single state machine:

State	Allowed commands	Transition
`CONNECTED`	`AUTH`, `EXIT`	Valid `AUTH` → `PASSWORD_VERIFIED`
`PASSWORD_VERIFIED`	`OTP`, `AUTH` (retry), `EXIT`	Valid `OTP` → `AUTHENTICATED`
`AUTHENTICATED`	`PCAP`, `MSG`, `EXIT`	`EXIT`, timeout, or disconnect → `CLOSED`
Any state	Anything else	Rejected with `ERROR`, logged; repeated failures close the connection

Why stateful.

1. Authentication is two steps (password, then OTP), so the server must remember which step the client has finished.
2. State lives on the server and ends with the connection. There is no token to steal and replay on a new connection.
3. Per-session counters (failed attempts, request count) support lockout and log correlation.
4. TCP is already connection-oriented, so this matches Labs 0–1 with the least extra machinery.

Why not stateless. A stateless design would need signed tokens, key management, and expiry handling on every request. That adds attack surface beyond Labs 0–5 and still would not stop replay without transport security.

Trade-offs accepted. The lab server called `listen(1)` and accepted one client, then exited. The final server accepts clients in a loop with a thread per connection, because the port scan sends probes to the server's own port and must not use up the only connection. Each connection costs memory, so there is a connection cap and idle timeout. A client that disconnects must log in again. Because the session is tied to the TCP connection and not to the user, an on-path attacker who hijacks an authenticated plain-TCP connection could issue commands, which is why TLS is the main production recommendation.

Part B – Adversarial Threat Model

Attacker model. (1) An unauthenticated client that can reach the server port; (2) an on-path observer or modifier of the TCP channel; (3) a local user who can read files on the server host; (4) an analyst who mistypes or misuses scan targets; (5) untrusted content in the PCAP file and in HTTP responses.

Legend. *Impl* = implemented in the submission. *Future* = proposed for production only. Where a mitigation changes the lab scripts, the lab behavior is named. Before submitting, edit the Mitigation column so it matches exactly what your final code does, because the exam requires the design and implementation to be consistent.

ID and area	Asset	Attack vector	Impact	Mitigation	Residual risk
T1 Sockets: malformed, oversized, or stalled frames, and scan probes	Server availability and memory	Huge or partial frames, invalid fields, idle connections; `nmap -sV` sends service probes and junk data to the server's own port	Denial of service; thread or memory exhaustion; crash; noisy logs	*Impl:* frame size cap, read-exactly loop with timeouts, validation of every field, per-connection exception handling, connection cap, idle timeout; accept loop with threads (the lab's `listen(1)` and single `accept()` would be used up by the first probe)	Floods from many sources can still exhaust resources; no network-level rate limiting. *Future:* firewall or reverse proxy

ID and area	Asset	Attack vector	Impact	Mitigation	Residual risk
T2 Sockets: sniffing and replay on plain TCP	Password, OTP, PCAP results	Passive capture, or on-path replay and modification of `AUTH`, `OTP`, and `PCAP` traffic	Credential disclosure; account takeover; result tampering	*Impl:* bind to `127.0.0.1`; reject reuse of an already-accepted OTP time-step. *Future:* TLS via `ssl`	No confidentiality or integrity on plain TCP. A captured OTP can still be used by an attacker who races the user inside the validity window. The password is sent in clear
T3 Sockets: unauthenticated or unauthorized commands	Protected functions (`PCAP`, `MSG`)	Sending `PCAP` before login, sending `OTP` before `AUTH`, skipping the OTP step, or commands built to match the lab's `startswith` check (for example `AUTHX`)	Unauthorized access to analysis data	*Impl:* exact command match; state-machine gate with default-deny; every violation answered with `ERROR` and logged	Authorization is tied to the TCP connection, so a hijacked authenticated connection is not detected. Single user role only
T4 Scanning: out-of-scope or excessive scanning	Third-party hosts; network availability; the analyst's compliance	The lab asks for any typed target; a mistyped host or very large range; changed nmap options	Unauthorized scanning, policy breach, network noise	*Impl:* target fixed to `127.0.0.1`; port numbers checked (1–65535, start ≤ end); 1000-port cap kept; nmap arguments fixed in code; scanner errors caught	The restriction is a code-level policy; someone with source access can remove it
T5 Scanning: inaccurate or disruptive results	Integrity of the assessment; the student server	Short timeouts or filtered ports give false results; `-sV` probes may confuse the server, or be logged as malformed requests	Wrong conclusions about exposure; log noise	*Impl:* compare results against a known open port (the server) and a known closed port; server answers junk input with `ERROR` and keeps running; scan range and options documented	Connect scan covers TCP only; service and version detection can be wrong

ID and area	Asset	Attack vector	Impact	Mitigation	Residual risk
T6 Authentication/MFA: brute force and user enumeration	User account	Repeated password guesses; guessing a 6-digit OTP; the lab returns different messages for an unknown user and a wrong password	Account takeover; confirmation of valid usernames	*Impl:* failed-attempt counter per connection and per username, lockout or delay, connection closed after the limit; one `ACCESS_DENIED` for both unknown user and wrong password; OTP checked as six digits; constant-time compare	Attacker can reconnect; in-memory counters reset on restart; lockout lets an attacker deny service to the real user. Distributed guessing is not stopped
T7 Authentication/MFA: credential store compromise	Password hashes; TOTP secrets	Local file read or backup leak; the lab keeps the `alice` password in source code and hashes it with unsalted SHA-256	Fast offline cracking; MFA bypass if the TOTP secret is taken	*Impl:* salted PBKDF2-HMAC-SHA256 with high iteration count; credentials moved out of source into a user store with restricted permissions; secrets never logged or sent after setup	The server must be able to recover the TOTP secret, so file theft defeats MFA. *Future:* encrypt secrets, use a secrets manager or HSM
T8 Authentication/Logging: sensitive data in console and logs	Passwords and OTPs	The lab prints the current OTP and the received OTP to the server console; logging raw requests; tracebacks returned to the client	Credential exposure from console, logs, or screenshots	*Impl:* those prints removed; log event type, time, address, username, and outcome only; field redaction; generic error text to the client	Usernames and addresses are still recorded (privacy). Log files have no tamper protection. *Future:* append-only remote log
T9 Scapy: path abuse and malformed or oversized PCAP	Server process; server filesystem	A client-supplied filename (`../` traversal); a crafted or very large capture that crashes Scapy or exhausts memory	Arbitrary file read; denial of service; parser bugs	*Impl:* client cannot choose a path (fixed server-side file); file size cap; one streaming `PcapReader` pass instead of loading the whole file with `rdpcap` ; parsing wrapped in error handling; analysis only, no replay or sending	Scapy dissector flaws remain possible and run inside the server process. *Future:* sandboxed low-privilege worker

ID and area	Asset	Attack vector	Impact	Mitigation	Residual risk
T10 Scapy: payload leakage and detection error	Confidentiality of capture contents; reliability of findings	Returning raw payloads over plain TCP; the rate rule (more than 20 packets from one source in 5 seconds) flags busy benign hosts and misses slow scans; spoofed sources grow the per-IP tracking table	Exposure of sensitive data; misleading conclusions; memory growth	*Impl:* return only counts, addresses, ports, and the evidence packet count (no payload bytes); threshold and window documented and justified; old timestamps removed; table size limited	Heuristic finds only fast, high-rate patterns; low-and-slow activity may evade it
T11 HTTP: unsafe targets, header and cookie misreadings, and over-interpreted findings	Analyst workstation; the test service; credibility of the report	Changed base URL pointing off-host; redirects to external sites; slow or oversized responses; a missing header or cookie flag treated as proof of a flaw	Unauthorized probing; client hang; false claims of exploitability	*Impl:* base URL fixed to the local test service (`127.0.0.1:8765`); read-only `GET` requests plus one `POST` used only to check the error reply; redirects not followed automatically and the `Location` target checked; 5-second timeout; no credentials or exploit payloads; every finding marked "manual review required"	Findings show indicators and configuration gaps, not proof of exploitability. Plain HTTP means TLS settings cannot be assessed

Coverage check

Required area	Threats
Sockets	T1, T2, T3
Scanning	T4, T5
Authentication / MFA	T6, T7, T8
Scapy-based analysis	T9, T10
HTTP assessment	T11

The highest residual risks, which Part G should discuss, are T2 (no confidentiality and a replay window on plain TCP), T7 (the TOTP secret must be recoverable by the server), and T6 (reconnect-based guessing and lockout abuse).

Part C – Protocol Design

The protocol keeps the text commands from the authentication lab (`AUTH`, `OTP`, `MSG`, `EXIT`), adds one command (`PCAP`), and fixes the lab's two weaknesses: messages had no boundaries (bare `recv(1024)`) and commands were matched by prefix.

C.1 Transport and framing

- **Transport:** TCP to `127.0.0.1:5050`, one connection per client session.
- **Frame:** a 4-byte big-endian length, then that many bytes of UTF-8 text. The receiver reads exactly 4 bytes, then exactly that many bytes, so a message can never be cut or merged.
- **Limits:** a frame is at most 65,536 bytes. A client request is at most 1,024 bytes; only the server's `PCAP` reply may be larger.
- **Timeouts:** 10 seconds to finish a started frame; 120 seconds idle before the server closes the session.

C.2 Message structure and commands

A request is `COMMAND|field|field`. The command is uppercase and matched exactly against the allowlist. The text is split into at most the number of fields the command defines, so a password that contains `|` stays intact.

Command	Format	Allowed in state	Success reply	Failure replies
`AUTH`	`AUTH|username|password`	`CONNECTED`, `PASSWORD_VERIFIED` (retry)	`OTP_REQUIRED`	`ACCESS_DENIED`; `ERROR|FORMAT`; `ERROR|LOCKED`
`OTP`	`OTP|123456`	`PASSWORD_VERIFIED`	`ACCESS_GRANTED`	`ACCESS_DENIED`; `ERROR|FORMAT`
`PCAP`	`PCAP`	`AUTHENTICATED`	`OK|<summary and alerts>`	`ERROR|AUTH_REQUIRED`; `ERROR|PCAP_UNAVAILABLE`
`MSG`	`MSG|text` (1–200 characters)	`AUTHENTICATED`	`OK|Message received from <username>`	`ERROR|AUTH_REQUIRED`; `ERROR|FORMAT`; `ERROR|TOO_LONG`
`EXIT`	`EXIT|`	any state	`OK|Goodbye`, then the server closes	none

Field rules: username 1–32 characters (letters, digits, `_`, `.`, `-`); password 1–128 characters; OTP exactly six digits.

C.3 Authentication and MFA sequence

1. The client connects. The server creates a session in `CONNECTED`.
2. The user types a username and password (password read with `getpass`). The client sends `AUTH|username|password`.
3. The server checks the field format and whether the username is locked. It then derives a PBKDF2-HMAC-SHA256 hash from the password and the user's stored salt and compares it with `hmac.compare_digest`. For an unknown username it runs the same computation against a dummy salt, so response time does not reveal which usernames exist.
4. Wrong password or unknown user: `ACCESS_DENIED` (the same reply for both), failure counter +1, event logged. Correct password: state becomes `PASSWORD_VERIFIED` and the reply is `OTP_REQUIRED`. The server never prints the expected OTP, unlike the lab.
5. The user reads the six-digit code from an authenticator app. The client sends `OTP|123456`.
6. The server refuses the request with `ACCESS_DENIED` unless the state is `PASSWORD_VERIFIED`. It then checks that the code is six digits and verifies it with `pyotp` (current 30-second step plus or minus one step). A time-step that was already accepted is rejected.
7. Valid code: state becomes `AUTHENTICATED`, reply `ACCESS_GRANTED`, event logged. Invalid code: `ACCESS_DENIED`, failure counter +1.
8. After 3 failed attempts on one connection the server closes it. After 5 failures for one username within 5 minutes, further `AUTH` requests for that username get `ERROR|LOCKED` for 60 seconds. These values are design choices to be tuned during testing.

C.4 Authorization rules for protected commands

- **Default-deny.** A command runs only if it is listed for the session's current state (table in C.2 and state table in A.4). The check happens before any field is parsed and before any handler runs.
- **Protected commands** are `PCAP` and `MSG`. They need `AUTHENTICATED`. Otherwise the reply is `ERROR|AUTH_REQUIRED` and the event `UNAUTHORIZED_COMMAND` is logged.

- **Order is enforced.** `OTP` is refused before a successful `AUTH`. `AUTH` is refused once the session is `AUTHENTICATED`, so a logged-in session cannot be silently switched to another account.
- **A failed `AUTH` resets the session** to `CONNECTED`, so an earlier password success cannot be reused with a later OTP attempt.
- **Local tools are not server-protected.** The client menu offers the port scan and HTTP check after login for a consistent workflow, but this is a usability rule, not a security control: both run on the client against localhost and never touch the server's protected functions.

C.5 Malformed request handling and error responses

Every request is treated as untrusted. Each row is a rule the server applies before any command logic runs.

Condition	How it is detected	Reply	Server action
Length header is 0 or above 65,536 (this is also what a text probe from `nmap -sV` looks like)	First 4 bytes parsed as a number	none	Close the connection; log `MALFORMED_FRAME` (no way to find the next frame boundary)
Frame started but not finished in 10 seconds, or the connection drops mid-frame	Read timeout or empty read	none	Close; log `INCOMPLETE_FRAME`
Payload is not valid UTF-8, is empty, or has no command	Decode and split	`ERROR|FORMAT`	Keep the session; count a protocol error

Condition	How it is detected	Reply	Server action
Unknown command, including near matches such as `AUTHX`	Exact match against the allowlist	`ERROR|UNKNOWN_COMMAND`	Log `UNKNOWN_COMMAND`; count a protocol error
Wrong number of fields, empty field, bad username characters, OTP not six digits	Field validation	`ERROR|FORMAT`	Count a protocol error
`MSG` text longer than 200 characters	Length check	`ERROR|TOO_LONG`	Count a protocol error
Command not allowed in the current state	State check (C.4)	`ERROR|AUTH_REQUIRED`, or `ACCESS_DENIED` for an out-of-order `OTP`	Log `UNAUTHORIZED_COMMAND`
Unexpected exception inside a handler	`try/except` around each request	`ERROR|SERVER`	Write the details to the server log only; close that connection; the server keeps running
5 protocol errors in a row on one connection	Per-session counter	`ERROR|FORMAT`	Close the connection; log `TOO_MANY_ERRORS`

Reply	Meaning
`ERROR|FORMAT|<text>`	Request could not be parsed or a field is invalid
`ERROR|UNKNOWN_COMMAND|<text>`	Command is not in the allowlist
`ERROR|AUTH_REQUIRED|<text>`	Command needs a state the session has not reached

Reply	Meaning
`ERROR|TOO_LONG|<text>`	A field exceeds its maximum length
`ERROR|LOCKED|<text>`	Username is temporarily locked after repeated failures
`ERROR|PCAP_UNAVAILABLE|<text>`	Capture file is missing, unreadable, or corrupt
`ERROR|SERVER|<text>`	Internal failure; no details given
`ACCESS_DENIED`	Login failed; the same reply for unknown user, wrong password, and wrong OTP

Error text is fixed and short. Replies never include stack traces, file paths, the submitted input, or any hint about whether a username exists.

C.6 Session termination

Trigger	Server behavior	Client behavior
User chooses exit	Receives `EXIT|`, replies `OK|Goodbye`, sets state to `CLOSED`, logs the logout, closes the socket	Prints the reply, closes the socket, returns to the shell
Client disconnects or resets	Empty read or `ConnectionResetError`: logs `CLIENT_DISCONNECTED`, discards the session, keeps listening for other clients (the lab ended the whole server here)	Not applicable
Idle for 120 seconds	Logs `IDLE_TIMEOUT` and closes the connection	Next send or receive fails and is reported
Too many failures or malformed frames	Closes the connection and logs the reason (C.3, C.5)	Shows "connection closed by server"
Server stops or crashes	Listener closes	Empty read or timeout is caught; the client prints "server disconnected" and exits cleanly instead of showing a traceback

Session state exists only in server memory and is destroyed with the connection. The password and OTP are used for the check and not stored in the session.

C.7 Replay and security limitations: implemented vs proposed

Protection or limitation	Status	What it does or why it is limited
Length-prefixed frames, size caps, timeouts	Implemented	Unambiguous messages; limits memory use and stalled reads
State machine with default-deny	Implemented	Blocks protected commands before login and out-of-order steps
Salted PBKDF2-HMAC-SHA256 password check	Implemented	Slows offline cracking of a stolen user store

Protection or limitation	Status	What it does or why it is limited
TOTP with `pyotp`; an accepted time-step is rejected a second time	Implemented	A captured OTP cannot be replayed after it has been used
Failure limits, lockout, uniform `ACCESS_DENIED`	Implemented	Slows guessing and hides which usernames exist
Bind to `127.0.0.1`; validated input; logging without passwords or OTPs	Implemented	Limits exposure and keeps credentials out of logs
Confidentiality of the channel	**Not provided**	Plain TCP: the password, OTP, and PCAP results are readable by anyone who can capture the traffic
Replay of the login messages	**Limited**	The OTP expires and cannot be reused, but a captured password stays valid, and an attacker who captures an OTP can use it first if they connect before the user finishes
Integrity and hijacking of an authenticated session	**Not provided**	Messages carry no signature or sequence number, so an on-path attacker can modify them or inject commands into a logged-in connection
TLS with the `ssl` module	Proposed	Adds confidentiality, integrity, and server authentication for the whole connection
Per-message sequence numbers or HMAC	Proposed	Detects replayed or modified messages inside a session
Challenge-response login (the password never crosses the wire)	Proposed	Removes the clear-text password from the channel
Encrypted TOTP secrets, persistent lockout counters, network rate limiting, tamper-evident remote logs	Proposed	Closes the residual risks listed for T6, T7, and T8

Update the Status column to match your final code. A control may appear as *Implemented* only if the code and the test evidence in Part F demonstrate it.

Part D – Component Design

D.1 Port scanner (`scanner.py` , from `namp_scanner.py`)

Aspect	Specification
Inputs	Start port and end port as integers from the menu. Target is fixed to `127.0.0.1` (the lab asked for any host). Nmap arguments are fixed to `-sT -sV`
Outputs	For each port nmap lists: port number, state, service name, product, version. A closing line gives the number of open ports and says whether the server's own port was found. When nothing is found: "No ports were discovered in the specified range."
Expected behavior	Validate the range, build the string `start-end` , run `python-nmap` , confirm the target appears in `all_hosts()` , then print each entry of `scanner[host]["tcp"]` in port order. The server's listening port must appear as open. Nmap lists closed ports one by one only for small ranges and summarizes them for large ones, so the module reports listed ports and states that unlisted ports are closed or filtered; the closed-port test uses a one-port range
Failure conditions	Empty, non-numeric, or out-of-range port (not 1–65535); start greater than end; more than 1000 ports; a target other than `127.0.0.1` ; nmap not installed or `PortScannerError` ; host missing from results; any other exception. Each prints a clear message and the scan does not continue
Security controls	Localhost-only target and a 1000-port cap; fixed arguments, so the user cannot add nmap scripts or options; no shell command is built from user input; results are shown locally and never sent to the server; the scan is run only on the authorized range

D.2 MFA module (`auth_session.py`, from `auth_server.py`)

Aspect	Specification
Inputs	Username and password from `AUTH`; six-digit code from `OTP`; the stored user record (salt, iteration count, password hash, TOTP secret, last accepted time-step); the current time; the session's state and failure counters
Outputs	`OTP_REQUIRED`, `ACCESS_GRANTED`, `ACCESS_DENIED`, or an `ERROR` code; an updated session state; one log event per decision
Expected behavior	`AUTH`: validate fields, check lockout, derive PBKDF2-HMAC-SHA256 with the stored salt, compare with `hmac.compare_digest`, move to `PASSWORD_VERIFIED`. `OTP`: require `PASSWORD_VERIFIED`, require six digits, verify with `pyotp.TOTP(secret).verify(code, valid_window=1)`, reject a time-step that was already accepted, then move to `AUTHENTICATED`. The TOTP secret is created once at setup, stored in the user store, and shown once as a provisioning URI so an authenticator app can be enrolled (the lab made a new random secret on every start)
Failure conditions	Empty or malformed fields; unknown user; wrong password; `OTP` sent before the password step; code not six digits; wrong, expired, or reused code; 3 failures on one connection (connection closed); 5 failures per username in 5 minutes (60-second lockout); missing or unreadable user store (the server refuses to start)
Security controls	Per-user random salt and a high iteration count (for example 600,000) replace the lab's unsalted SHA-256; constant-time comparison; one `ACCESS_DENIED` for every login failure, with a dummy hash for unknown users; no password, OTP, or secret is printed or logged; the user store is read once at startup and is not part of the source code; default-deny for every later command

D.3 HTTP checker (`http_checker.py`, from `http_scanner.py` and `vulnerability_detector.py`)

Aspect	Specification
Inputs	Fixed base URL `http://127.0.0.1:8765`, the supplied `test_website.py` (the labs' port 3000 and search endpoint do not exist on it). Fixed paths: `/`, `/health`, `/redirect`, and one unknown path for the 404 case. One `POST` to `/` to check the unsupported-method reply. Header list to check: `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`. Timeout 5 seconds
Outputs	One console line per request (method, path, status, length, content type). A findings list with category, path, observation, and review status. Two CSV files: `http_scan_results.csv` (observations) and `http_findings.csv` (findings). A summary that ends with a reminder that findings are not proof of exploitability

Aspect	Specification
Expected behavior	Step 1: request each path with `allow_redirects=False` and record status, length, content type, and `Server` header. Step 2: from the `/` response, report each missing security header (the test site leaves several out on purpose). Step 3: read the `Set-Cookie` header (`training_session=demo123; Path=/`) and report the missing `HttpOnly`, `Secure`, and `SameSite` attributes. Step 4: `/redirect` should return 302 with `Location: /`; record it and confirm the target stays on the same host. Step 5: report the `Server` banner (`FSCT8561LabHTTP/1.0`) as informational disclosure, and that the site uses plain HTTP. Step 6: the `POST` is expected to get an error such as 501; record it as the unsupported-method result. Duplicate alerts are suppressed with a seen-set. The labs' reflected-marker and SQL-error probes are dropped: the test site has no search endpoint or reflected input, and the exam forbids exploitation
Failure conditions	Connection refused or timeout (for example `test_website.py` is not running, or the port is wrong): the request is recorded with status `ERROR`, a message is printed, and the remaining checks continue. If every request fails, the summary says "target unreachable, no conclusion" instead of "no indicators found" (the lab printed the second message even when the site was down). A path returning a different status than expected, a response without `Set-Cookie`, or a non-text body is recorded as an observation, not a crash. A CSV that cannot be written is reported

Aspect	Specification
Security controls	URL fixed to `127.0.0.1:8765`; read-only requests; redirects never followed automatically; timeout on every request; no credentials, payloads, or exploit strings sent; CSV stores observations, not full response bodies; every finding is worded as a configuration observation that needs manual review, because a missing header or cookie flag shows a protection is absent, not that an attack works, and the cookie here holds training data only

D.4 PCAP analyzer (`pcap_analyzer.py`, from `traffic_sniffer.py` and `anomaly_detector.py`)

Aspect	Specification
Inputs	The `PCAP` command from an `AUTHENTICATED` session (no arguments); the fixed server-side path of `FSCT8561_Operation_Sentinel_Traffic.pcap`; rule settings: 5-second window and a threshold of more than 20 packets per source IP
Outputs	`OK|` followed by text: total TCP packets, total UDP packets, total IPv4 TCP/UDP packets, the first 20 TCP/UDP packets (source, destination, protocol, source port, destination port), one `ALERT` line per suspicious source IP with its packet count, and the number of suspicious IPs. Output is capped to fit one frame

Aspect	Specification
Expected behavior	One streaming pass with `PcapReader`. For each IPv4 TCP or UDP packet: count its protocol, record it if fewer than 20 packets have been listed, add its timestamp to that source IP's window, drop timestamps older than 5 seconds, and raise one alert for the IP the first time its window holds more than 20 packets. Justification for the rule: a host that sends dozens of packets within seconds is far above normal conversational traffic and matches scanning or automated beaconing. The threshold is checked against the supplied capture, and the alerting packet counts are cited as evidence in Part G
Failure conditions	File missing, unreadable, or larger than the size cap; corrupt or truncated capture (Scapy raises an exception); non-IP packets (skipped, not an error); result too large for a frame (truncated with a notice); request sent outside `AUTHENTICATED` (`ERROR|AUTH_REQUIRED`). File problems return `ERROR|PCAP_UNAVAILABLE` and details go to the server log only
Security controls	Path fixed on the server, never taken from the client, so no path traversal; analysis is read-only and sends or replays no packets; streaming read instead of loading the file twice; only headers, counts, and ports are returned, never payload bytes; per-IP window uses a queue, and the number of tracked IPs is limited; every request is logged with the username

D.5 Sensitive information deliberately not collected or logged

Category: authentication secrets, meaning plaintext passwords and one-time passcodes (and, with them, TOTP secrets).

The application checks these values in memory and discards them. The log records only the time, remote address, username, event type (`LOGIN_FAILED`, `OTP_FAILED`, `LOGIN_SUCCESS`, `UNAUTHORIZED_COMMAND`, and similar), and the outcome. Neither the

server nor the client writes the secret values to a log, a CSV file, or the console.

Why this choice:

1. A log file is read by more people and copied to more places (backups, screenshots, bug reports) than the user store, so a password in it would be a second, weaker credential store.
2. A stored password is useful to an attacker long after the event, and users reuse passwords on other systems.
3. The security decision (success or failure, who, when, from where) is fully recorded without the secret, so investigators lose nothing they need.
4. The lab server printed the current and the received OTP to its console. That helped debugging but would leak a valid code to anyone watching the terminal, so the final version removes it.